ROS Control Example 7:

Full tutorial with a 6 DOF robot

{ Yaml Parameter config
{ URDF

hardware { state_interface → controller
command_interface ←

↓ ↓

class HardwareInterface ControllerInterface.

main program : ros2_control_node

read → update → write

URDF

link, joint, ros2_control

↓ ↓ ↓

(parts) TF ✓ hardware / controller config

↓

export to .stl/.dae files

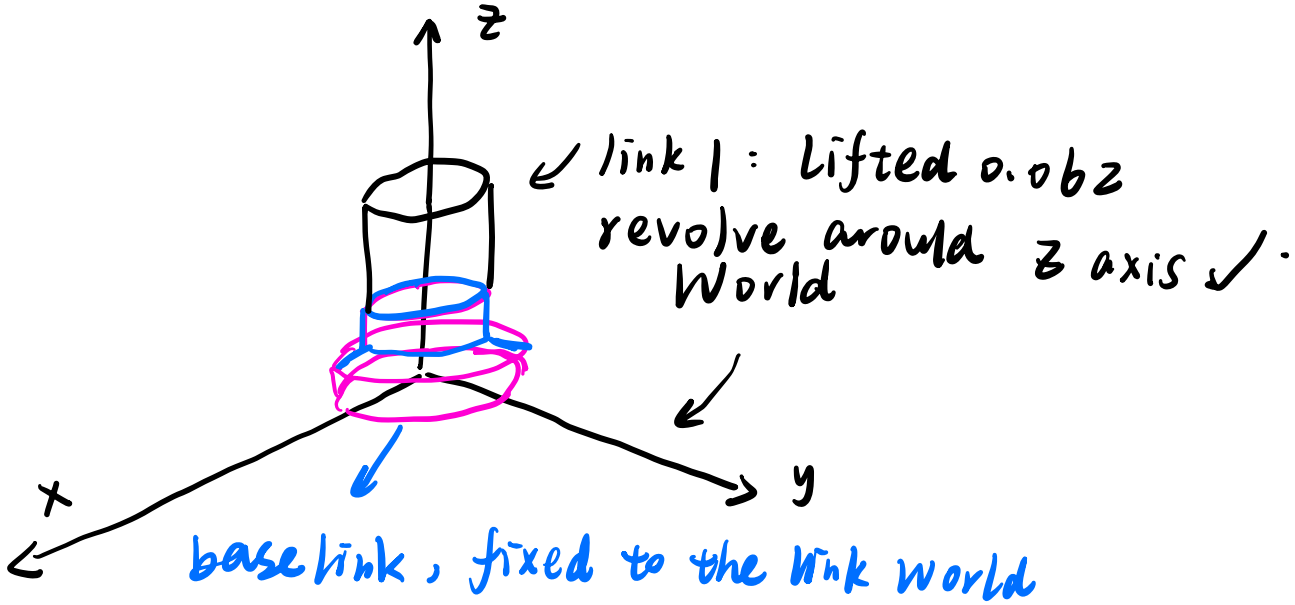
coarse meshes / visual purpose

解读：

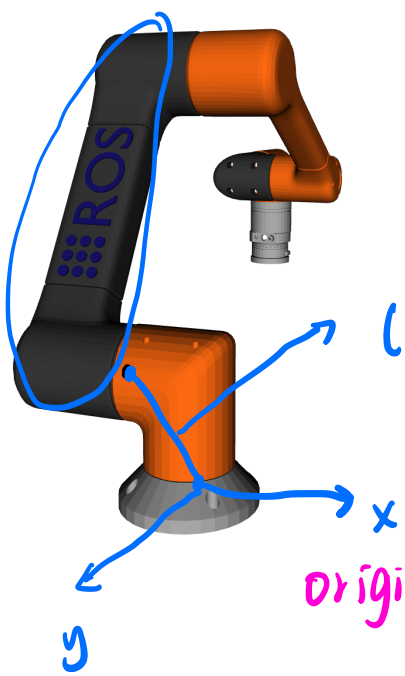
part 1: URDf

① creat link，用了 macro ✓

② set joints

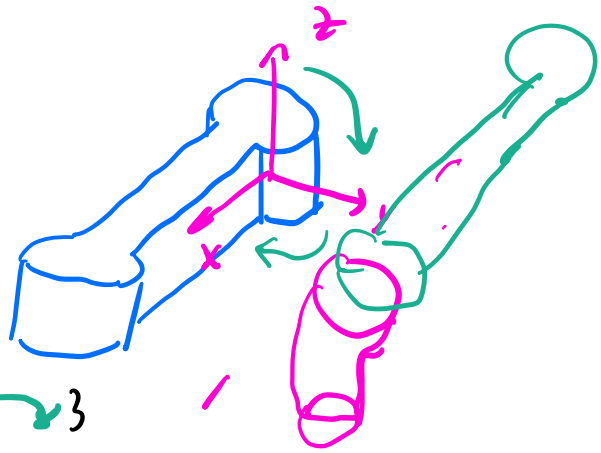


③ analyze translational / rotated offset



平移

(-0.1, 0, 0.18)



1 → 2 → 3

origin, rpy $(-\frac{\pi}{2}, -\frac{\pi}{3}, \frac{\pi}{2})$ → 相对于

绝对时会 X parent link frame

in <ros2_control name="0" type="system / actuator / sensor">

this should your PACKAGE name

① for a system (our case)

- you have many <joint> → eg "joint1"
  with their name +
  < command / state interface and name > → eg "position"
  - <param name="min" / "max"> value </param>

- This design: joints in /description have identical names in /ros2_control.

ensure realtime Simulation in Gazebo.
better virtual simulation with CLI enabled.

Format For .yaml <- For py launch script.

Controller_manager ✡ necessary

.ros parameter :
. update rate : 10 #Hz

{ your controllers' name :
their type: (your customized classes)

in .yaml file. you should specify controller's claimed resource ⇒ launch.py can instantiate them

part 2 CONTROLLER TO SPAWN

rbbot_controller: ← name of your Controller

ros_parameters: {
joints: - joint1
- joint2
} joint names ⇒ THEY SHOULD ALIGN WITH URDF

command interface ✓
- position "name"

State interface ✓
- velocity "name"

Hardware Interface.

You must overwrite

- on_init ✓
- export * interface x2. ✓
- read / write ✓

"in humble (not) in Jazzy"

Question: How does the config info in URDF parsed provided and stored in a structured way in your class?

Answer:

The CM does this as soon as it's created!

<prior to controllers!>

Store the joints information <= in a HardwareInfo Struct

① parse URDF

for lines: <ros2_control>

② use <hardware>

<plugin> Your Package/ Your Cuscomized class </plugin>

③ call loader() of Resoure Manager → memory allocated

↓

Further: on_init of your class!

✓ (hardware ready and alive)

1. name ✓
2. type ✓
3. class (your derived class's name)
4. joints / sensors
   - name of the component to "joint_1"

b its type "joint/GPIO/sensor"

c command/state_interface
{ name : "effort, velocity, position"
  min/max

(注意，认真的同学会发现这里真的和
☆ urdf 文件中 ros2_control 所给信息一一对应！)

---

on_init 主要把

① std::vector<double> / double joint_position_
这种由 driver 及其附属程序直接修改的变量
初始化好！

② (For this case)
joint_name
↓
use for iteration
for(& joint_name : joint_interfaces)
"position"/"velocity"

一个字典 { position → joint0 ~ joint5
velocity → joint0 ~ joint5

export_state_interfaces()
↓
① return a vector<CommandInterface> of state_interface
it's defined in hardware_interface, based on read-only handle. so you can't modify after constructed.

② use emplace back to construct & push a state_interfac
(iterate for 12 times) : 6 joints x ["pos", "vel"] ✓

- The export-state-interface is called when CM calls "read()" in the control loop
- The interface is further loaned by RM (wrap it!) and dispense to multiple controllers (remember state If is not exclusively accessed) ☆

↓ first hardware shall read()
and update the "values" of all interfaces! △

In contrast, the export-command_interface

↙

create & construct command_interfaces

↓ for controller to access & modify

update() → the controllers DIRECTLY
overwrite the double-joint_position_comvad
velocity

(That's feasible, as you transmit a
*value-ptr (be specific, you give & of them)

Further in write() call of CM,
the double command values will be translated and
send to hardware unsigned long/int in CAN protocol.

(But Here, our write does nothing as we don't really have a hardware to control, example 7)

Export your class as PLUGINLIB to finalize your .cpp file.

alignment with urdf + .xml files ①

Cmakelise file ②

In your hardware's cpp file, lase line (as always):

PLUGINLIB_EXPORT_CLASS (class_type , base_class_type)

Be coherent with <class> </class> in XML

This file allows ROS2 to discover/load plugin

<library path = "...")

install/lib

where the (.so) dynamic library file of your class exist.

So in Cmakelise → add_library (path) SHARED .cpp file

↳ pluginlib_export_plugin_description_file

Chardware_interface NAME of your.xml file)

ros 就是这么麻烦。

- Writing a controller

A controller has a finite set of states:

- Unconfigured
- Inactive
- Active → main control loop
- Finalized

on_*

↓

optional to override.

In our case

on_activate, on_deactivate
on_configure are used.

Your customized controller MUST implement:

- command_interface_configuration
- state_interface_configuration
- update
- on_init

As a beginner's note, I'll explain this in sequence of the execution.

Like a state machine / lifecycle node,
it has a pattern of doing specific task in each state:

I. Unconfigured

① construction : by CM with pluginlib interface

Initialize { all it's member
member / method from ControllerInterfaceBase.

☆ node_ (a lifecycle node) gives a sharedptr to it.
with: the NAME of joints, command_interfaces and state_interfaces
(They're vector of strings)

② call on_init()

- declare & read parameter using auto_declare from the base class
  → returns the NAME of joints, command_interface and state_interfaces FROM node_
- sets up size of point_interp_ (internal DS)

③ { Command / State } _interface_configuration

generate a struct of type InterfaceConfiguration conf

{ type (2, 1, 0)

- 2 → 没Hardware, 打光棍 NONE
- 1 → 一夫一妻 INDIVIDUAL
- 0 → 海王 All

vector<string> names

joint_name + "/" + interface type.

eg. joint1/velocity

conf → The CM uses this list to check the availability and permissions of interfaces in the underlying Hardware

Interfaces : For activate()

I. inactive

on_configuration → set up non-real time communication (Not in control loop)

sets up subscriber through CM node!

IN OUR CASE:

we also.

Ⅲ Activate State

① CM tries to obtaining & loaning the requested interf handles from < RM or Hardware interface

success / fail

↓ instruct

② the Controller call assign_interface method so obtain the loaned handles

(by std::forward of a rvalue)

They' become member of our class: (type, vector<hadware_interface>)

command_interfaces_ / state_interfaces_

③ After the ownership transfer is done, call on_activate().

↓

1° clears joint_position_ < velocity / command_interface_ < state

2° by convention, define a map + sorts and Stores the loaned interfaces

④ updates

in real_time loop, executed frequently

(i) reads the lastest command from outside node

(if You publish/subscribe to sth)

(ii) compute for the desired joint.position/velocity command ✓

(iii) as the loaned interfaces are actually alias, overwrite their command interface, using method set_value()

↳ the hardware // later call write() method.

(iv) deactivate

use release_interfaces() of base class to clear them and return the claimed handles! ✓

BONUS: launch file & control logic